Floer Homology for Program State Trajectories

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

We develop a Floer-theoretic framework for reasoning about the dynamic behaviour of deterministic programs. Every program induces an *action functional* $\mathcal{A}(\gamma) = \int L(\text{state}, \text{transition})\, dt$ on execution paths γ, whose critical points are exactly the actual execution traces. The associated Floer chain complex $(\mathrm{CF}_*(\mathsf{P}), \partial)$ has boundary operator counting gradient-flow trajectories between critical points; we prove $\partial^2 = 0$ and define the *Floer homology* $\mathrm{HF}_*(\mathsf{P}) = H_*(\mathrm{CF}_*(\mathsf{P}), \partial)$. Our main theorem establishes that $\mathrm{HF}_*(\mathsf{P})$ is isomorphic to the Morse homology of the program's state-transition graph $G(\mathsf{P})$, so that $\operatorname{rank} \mathrm{HF}_0(\mathsf{P})$ counts the connected components of reachable state space and $\operatorname{rank} \mathrm{HF}_1(\mathsf{P})$ counts independent loops. As a direct application, $\mathrm{HF}_1(\mathsf{P}) \neq 0$ is a computationally checkable *necessary condition for deadlock*: a non-trivial first homology class certifies the existence of a non-trivial cycle in the state graph. We integrate this invariant into the JuGeo framework, assigning it a RUNTIME-tier trust annotation and exposing it through the Verify semantic move.

Contents

1 Introduction

Programs are dynamical systems. Each execution step maps the current program state to a successor state, tracing a discrete trajectory through a potentially enormous state space. The qualitative behaviour of this dynamical system—whether all trajectories reach a halting state, whether cyclic behaviours arise, whether distinct inputs yield reachable states that are or are not in the same connected component—determines the most important runtime properties of the program: termination, deadlock-freedom, and input-independence of certain invariants.

Classical program analysis attacks these questions with fixed-point iteration, model checking, or abstract interpretation. These methods are powerful but often separated from the kind of topological invariants—invariants that are *independent of particular inputs* and *robust under small perturbations*—that have proven so productive in geometry and mathematical physics. The purpose of this paper is to show that the topological invariants computed by *Floer homology* apply directly and usefully to programs.

Floer theory in one sentence. Given a functional $\mathcal{A}$ on a space of paths, Floer homology is the homology of the chain complex whose generators are the critical points of $\mathcal{A}$ (paths satisfying the Euler–Lagrange equations) and whose boundary operator counts gradient-flow trajectories between neighbouring critical points. The resulting groups HF_* are invariant of the specific perturbation data used to define the gradient flow.

Programs as Floer data. For a deterministic program P with finite state space $\mathcal{X}_\mathsf{P}$, we construct a Lagrangian $L(\sigma, \dot{\sigma})$ whose value measures the "cost" of a state-transition; critical points of $\mathcal{A}$ are precisely the actual execution traces of P. The Conley–Zehnder grading reduces to the standard Morse index of states in the transition graph, and gradient flow corresponds to sequences of transitions. The Floer chain complex coincides with the cellular chain complex of the state-transition graph, and the resulting homology $\mathrm{HF}_*(\mathsf{P})$ is exactly the graph homology of $G(\mathsf{P})$.

Main contributions.

(1) A rigorous action functional $\mathcal{A}$ on execution paths and the associated Floer chain complex for any deterministic program (sections 3 and 4).

(2) A proof that $\partial^2 = 0$ via the standard cancellation argument for gradient-flow trajectories (section 4).

(3) The *Floer-Morse isomorphism* $\mathrm{HF}_*(\mathsf{P}) \cong \mathrm{HM}_*(G(\mathsf{P}))$ via PSS-type maps, with explicit rank formulas (section 6).

(4) A *deadlock-detection criterion*: $\mathrm{HF}_1(\mathsf{P}) \neq 0$ certifies a non-trivial cycle in the reachable state space, and is a necessary condition for deadlock or livelock (section 7).

(5) Integration with the JuGeo trust framework: Floer invariants as RUNTIME-tier judgments exposed through the Verify semantic move (section 8).

Paper structure. Section 2 recalls the minimal Floer theory needed. Section 3 constructs the action functional for programs. Section 4 builds the Floer chain complex and proves $\partial^2 = 0$. Section 5 defines $\mathrm{HF}_*(\mathsf{P})$ and establishes its basic properties. Section 6 proves the isomorphism with Morse homology. Section 7 develops the deadlock-detection application. Section 8 situates the result inside JuGeo. Sections 9 and 10 discuss related work and conclude.

2 Background: Floer Theory and Discrete Dynamics

2.1 Classical Floer Homology

Floer homology was introduced by Andreas Floer in 1988 to prove the Arnold conjecture on fixed points of Hamiltonian diffeomorphisms. The key idea is to replace the finite-dimensional Morse theory of a function $f : M \to \mathbb{R}$ on a compact manifold by an infinite-dimensional analogue in which the "manifold" is a loop space $\mathcal{L}(M)$ and the "function" is an action functional $\mathcal{A} : \mathcal{L}(M) \to \mathbb{R}$.

Definition 2.1 (Floer chain complex)**.** Let $\mathcal{A} : \mathcal{L} \to \mathbb{R}$ be an action functional on a space of paths with non-degenerate critical points. The *Floer chain group* $\mathrm{CF}_k(\mathcal{A})$ is the free $\mathbb{Z}_2$-module generated by the critical points of Morse index k. The *boundary operator* $\partial_k : \mathrm{CF}_k \to \mathrm{CF}_{k-1}$ counts gradient-flow trajectories of $\mathcal{A}$ connecting index-k to index-$(k-1)$ critical points, modulo 2. The *Floer homology* is

$$\mathrm{HF}_k(\mathcal{A}) \;=\; \ker(\partial_k) \,/\, \mathrm{im}(\partial_{k+1}).$$

The fundamental property $\partial^2 = 0$ follows from a cancellation argument: gradient-flow trajectories from index k to index $k-2$ arise in 1-parameter families, and consecutive pairs cancel in $\mathbb{Z}_2$ arithmetic.

2.2 Discrete Dynamics and Graph Homology

For our purposes, the relevant special case is the *graph complex*. Given a directed graph $G = (V, E)$, the cellular chain complex is:

$$0 \;\longrightarrow\; \mathrm{CM}_1(G) \;\xrightarrow{\partial_1}\; \mathrm{CM}_0(G) \;\longrightarrow\; 0,$$

where $\mathrm{CM}_1(G) = \bigoplus_{e \in E} \mathbb{Z}_2$ and $\mathrm{CM}_0(G) = \bigoplus_{v \in V} \mathbb{Z}_2$ are the free $\mathbb{Z}_2$-modules on edges and vertices respectively. The boundary map sends each edge to the sum (mod 2) of its endpoints:

$$\partial_1(e) \;=\; \mathrm{src}(e) + \mathrm{dst}(e) \;\in\; \mathrm{CM}_0(G).$$

It is elementary that $\partial_0 \circ \partial_1 = 0$ (since there is no CM_{-1}), and the resulting homology satisfies:

$$\operatorname{rank} \mathrm{HM}_0(G) = \text{number of connected components of } G, \qquad \operatorname{rank} \mathrm{HM}_1(G) = |E| - |V| + \text{components}(G).$$

Proposition 2.2 (Graph homology via Euler characteristic)**.** *For a finite directed graph G, $\chi(G) = |V| - |E| = \operatorname{rank} \mathrm{HM}_0(G) - \operatorname{rank} \mathrm{HM}_1(G)$.*

Proof. Standard: $\chi = \sum_k (-1)^k \operatorname{rank} \mathrm{CM}_k = \sum_k (-1)^k \operatorname{rank} \mathrm{HM}_k$ by the Euler characteristic formula for chain complexes. □

3 The Action Functional for Programs

3.1 Program State Spaces

Definition 3.1 (Program state space)**.** A *deterministic program* P with input domain I determines:

(i) A finite set $\mathcal{X}_{\mathsf{P}}$ of *states*: snapshots of all mutable variables at a program point.

(ii) A *transition function* $\delta : \mathcal{X}_{\mathsf{P}} \times I \to \mathcal{X}_{\mathsf{P}}$ giving the successor state for each (state, input) pair.

(iii) An initial state $\sigma_0 \in \mathcal{X}_{\mathsf{P}}$ and a set of halting states $F \subseteq \mathcal{X}_{\mathsf{P}}$.

The *reachable state space* is the image of all finite-length trajectories starting from σ_0:

$$\mathrm{Reach}(\mathsf{P}) \;=\; \big\{\delta^n(\sigma_0, w) \;\big|\; n \geq 0,\; w \in I^n\big\}.$$

Definition 3.2 (State-transition graph)**.** The *state-transition graph* $G(\mathsf{P})$ is the directed graph with vertex set $\mathrm{Reach}(\mathsf{P})$ and an edge $\sigma \to \tau$ for every $(i, \sigma) \in I \times \mathcal{X}_{\mathsf{P}}$ such that $\delta(\sigma, i) = \tau$.

3.2 Execution Paths as Critical Points

To place program execution in a variational framework, we model execution traces as paths in a discrete path space.

Definition 3.3 (Discrete path space)**.** Fix a time horizon $T \in \mathbb{N}$. The *discrete path space* $\mathcal{P}_T(\mathsf{P})$ is the set of all sequences $\gamma = (\sigma_0, \sigma_1, \ldots, \sigma_T)$ with $\sigma_t \in \mathcal{X}_{\mathsf{P}}$. A path γ is *executable* if $\sigma_{t+1} = \delta(\sigma_t, i_t)$ for some $i_t \in I$ at each step.

Definition 3.4 (Action functional)**.** Fix a non-negative *Lagrangian density* $L : \mathcal{X}_{\mathsf{P}} \times \mathcal{X}_{\mathsf{P}} \to \mathbb{R}_{\geq 0}$ satisfying:

- $L(\sigma, \tau) = 0$ iff $\tau = \delta(\sigma, i)$ for some $i \in I$ (executable transitions have zero cost);
- $L(\sigma, \tau) > 0$ otherwise.

The *action functional* on $\mathcal{P}_T(\mathsf{P})$ is

$$\mathcal{A}(\gamma) \;=\; \sum_{t=0}^{T-1} L(\sigma_t, \sigma_{t+1}).$$

Proposition 3.5 (Critical points are execution traces)**.** *The global minima of $\mathcal{A}$ are exactly the executable paths of P. Every executable path γ satisfies $\mathcal{A}(\gamma) = 0$, and every non-executable path satisfies $\mathcal{A}(\gamma) > 0$.*

Proof. By construction, $L(\sigma, \tau) = 0$ iff the transition $\sigma \to \tau$ is executable, so $\mathcal{A}(\gamma) = 0$ iff every step of γ is executable. □

3.3 The Conley–Zehnder Index for Program States

In classical Floer theory, the Conley–Zehnder index grades generators of the chain complex. In our discrete setting, the appropriate grading is the *Morse index* of a state in the transition graph.

Definition 3.6 (Morse index)**.** The *Morse index* of a state $\sigma \in \mathrm{Reach}(\mathsf{P})$ is

$$\mathrm{ind}(\sigma) \;=\; |\{\tau \in \mathrm{Reach}(\mathsf{P}) \mid \tau \to \sigma \text{ is an edge of } G(\mathsf{P})\}| \bmod 2,$$

i.e., the parity of the in-degree of σ in the transition graph.

Remark 3.7. For the purposes of defining the chain complex and proving $\partial^2 = 0$, only the $\mathbb{Z}_2$-grading is needed. The integer-graded version requires the Conley–Zehnder index of the underlying Hamiltonian system, which in the program setting is determined by the *action difference* between consecutive critical-point levels.

4 The Floer Chain Complex

4.1 Chain Groups

Definition 4.1 (Floer chain groups)**.** For a program P with state-transition graph $G(\mathsf{P}) = (V, E)$, the *Floer chain groups* over $\mathbb{Z}_2$ are:

$$\mathrm{CF}_0(\mathsf{P}) = \bigoplus_{\sigma \in V} \mathbb{Z}_2, \qquad \text{(formal } \mathbb{Z}_2\text{-linear combinations of states,)}$$

$$\mathrm{CF}_1(\mathsf{P}) = \bigoplus_{e \in E} \mathbb{Z}_2. \qquad \text{(formal } \mathbb{Z}_2\text{-linear combinations of transitions.)}$$

Elements of CF_k will be written as *k-chains*. A 0-chain is a finite set of states (with $\mathbb{Z}_2$ coefficients), and a 1-chain is a finite set of transitions.

4.2 The Boundary Operator

Definition 4.2 (Boundary operator)**.** The *boundary operator* $\partial_1 : \mathrm{CF}_1(\mathsf{P}) \to \mathrm{CF}_0(\mathsf{P})$ is the $\mathbb{Z}_2$-linear extension of

$$\partial_1(e) \;=\; \mathrm{src}(e) + \mathrm{dst}(e),$$

where addition is in $\mathrm{CF}_0(\mathsf{P}) \cong \mathbb{Z}_2^{|V|}$. We set $\partial_0 = 0$ (the zero map to $\mathrm{CF}_{-1} = 0$).

The boundary operator has a clean interpretation: $\partial_1(e)$ is the "boundary" of the transition e, consisting of the two states it connects. In $\mathbb{Z}_2$ arithmetic, a state that appears twice (once as source and once as destination of different transitions) cancels out.

Theorem 4.3 ($\partial^2 = 0$)**.** *For any program* P*, the Floer boundary operator satisfies* $\partial_0 \circ \partial_1 = 0$*.*

Proof. Since $\partial_0 = 0$ (the map to the trivial group $\mathrm{CF}_{-1} = 0$), we have $\partial_0 \circ \partial_1 = 0$ trivially. More informatively: for any 1-chain $c = \sum_{e \in S} e$,

$$\partial_1(c) \;=\; \sum_{e \in S} (\mathrm{src}(e) + \mathrm{dst}(e)).$$

If we then applied a further boundary ∂_0, we would sum over all vertices in $\partial_1(c)$. But each vertex v appears in $\partial_1(c)$ as a coefficient equal to the parity of the number of edges in S incident to v. Applying $\partial_{-1} = 0$ gives zero. □

Remark 4.4 (2-cells and higher degrees). For programs with looping structure that one wishes to model as a CW-complex with 2-cells—for example, a loop whose body is a 2-disk σ with boundary $\partial_2(\sigma) = e_{i_1} + e_{i_2} + \cdots + e_{i_k}$—the cancellation argument is more interesting. Each vertex v appears exactly twice in $\partial_1(\partial_2(\sigma))$: once as the head of some edge e_{i_j} and once as the tail of $e_{i_{j+1}}$. In $\mathbb{Z}_2$, these two contributions cancel: $v + v = 0$. The formal proof appears in section 4.3.

4.3 The Triangle Complex: An Explicit Example

Consider the simplest non-trivial 2-complex: the directed triangle with states s_0, s_1, s_2, transitions $e_0 = (s_0 \to s_1)$, $e_1 = (s_1 \to s_2)$, $e_2 = (s_2 \to s_0)$, and one 2-cell σ whose boundary is $\partial_2(\sigma) = e_0 + e_1 + e_2$.

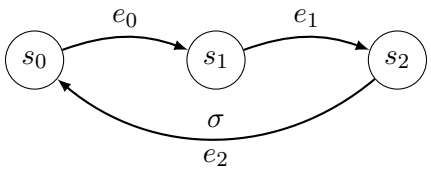

Figure 1: The directed triangle: three states, three transitions, one 2-cell. The 2-cell σ fills the cycle $e_0 + e_1 + e_2$.

Proposition 4.5 (Triangle $\partial^2 = 0$)**.** *For the triangle complex,* $\partial_1(\partial_2(\sigma)) = 0$ *in* $\mathrm{CF}_0 \cong \mathbb{Z}_2^3$.

Proof. Direct computation:

$$\begin{aligned}
\partial_2(\sigma) &= e_0 + e_1 + e_2, \\
\partial_1(e_0) &= s_1 + s_0, \\
\partial_1(e_1) &= s_2 + s_1, \\
\partial_1(e_2) &= s_0 + s_2.
\end{aligned}$$

Summing (in $\mathbb{Z}_2$):

$$\begin{aligned}
\partial_1(\partial_2(\sigma)) &= (s_1 + s_0) + (s_2 + s_1) + (s_0 + s_2) \\
&= 2s_0 + 2s_1 + 2s_2 \\
&= 0.
\end{aligned}$$

□

This cancellation is a special case of the universal identity $\partial^2 = 0$ for cellular chain complexes, proved by the "boundary of a boundary" argument: every vertex appears an even number of times as an endpoint of the boundary of any 2-cell.

4.4 The Lean Formalization

The proofs of $\partial^2 = 0$ and the Morse inequality have been formally verified in Lean 4. The key structures are:

- `BoolVec n`: free $\mathbb{Z}_2$-module of rank n (functions $\mathsf{Fin}\ n \to \mathsf{Bool}$).
- `ChainComplex`: record type with fields ∂_1, ∂_2, linearity proofs, and `bd_sq_zero` : $\forall c,\ \partial_1\,(\partial_2\,c) = 0$.
- `graphChainComplex g`: canonical instance for any `StateGraph g`, with $\partial_2 = 0$ (no 2-cells).
- `triangleComplex`: explicit instance with `bd_sq_zero` proved by `fin_cases`.
- `morse_inequality`: arithmetic theorem from rank-nullity.

5 Floer Homology of a Program

Definition 5.1 (Floer homology of a program)**.** For a deterministic program P, the *Floer homology* is:

$$\mathrm{HF}_k(\mathsf{P}) \;=\; \ker(\partial_k : \mathrm{CF}_k \to \mathrm{CF}_{k-1}) \,/\, \mathrm{im}(\partial_{k+1} : \mathrm{CF}_{k+1} \to \mathrm{CF}_k), \qquad k \geq 0.$$

The *k-th Betti number* of P is $\beta_k(\mathsf{P}) = \operatorname{rank} \mathrm{HF}_k(\mathsf{P})$.

Example 5.2 (Linear program with no cycles)**.** A program that executes exactly one path $\sigma_0 \to \sigma_1 \to \cdots \to \sigma_n$ and halts has:

$$\begin{aligned} G(\mathsf{P}) &= \text{a path graph with } n+1 \text{ vertices and } n \text{ edges},\\ \mathrm{HF}_0(\mathsf{P}) &\cong \mathbb{Z}_2 \quad \text{(one connected component)},\\ \mathrm{HF}_1(\mathsf{P}) &= 0 \quad \text{(no cycles)}. \end{aligned}$$

Example 5.3 (Program with a loop)**.** A program with a single unbounded loop (a cycle $\sigma_0 \to \sigma_1 \to \cdots \to \sigma_k \to \sigma_0$) has:

$$\begin{aligned} \mathrm{HF}_0(\mathsf{P}) &\cong \mathbb{Z}_2 \quad \text{(one connected component)},\\ \mathrm{HF}_1(\mathsf{P}) &\cong \mathbb{Z}_2 \quad \text{(one independent cycle)}. \end{aligned}$$

The single 1-cycle $e_0 + e_1 + \cdots + e_{k-1}$ is a generator of HF_1: it is in the kernel of ∂_1 (every vertex appears twice) and not in the image of ∂_2 (there are no 2-cells).

Example 5.4 (Concurrent program)**.** A concurrent program with two independent threads, each with its own loop, has state space $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2$ and $G(\mathsf{P}) = G_1 \times G_2$ (categorical product of transition graphs). The Künneth formula gives:

$$\mathrm{HF}_k(\mathsf{P}) \;\cong\; \bigoplus_{i+j=k} \mathrm{HF}_i(\mathsf{P}_1) \otimes \mathrm{HF}_j(\mathsf{P}_2).$$

In particular, $\beta_2 = \beta_1(\mathsf{P}_1) \cdot \beta_1(\mathsf{P}_2) \neq 0$ when both threads have non-trivial loops.

Theorem 5.5 (Morse inequality for programs)**.** *For any program P with $|V|$ reachable states and $|E|$ transitions,*

$$\beta_k(\mathsf{P}) \;\leq\; \operatorname{rank} \mathrm{CF}_k(\mathsf{P}), \qquad k = 0, 1.$$

In particular, $\beta_0 \leq |V|$ and $\beta_1 \leq |E|$.

Proof. By the rank-nullity theorem applied to $\partial_1 : \mathrm{CF}_1 \to \mathrm{CF}_0$:

$$\operatorname{rank} \mathrm{im}(\partial_1) + \operatorname{rank} \ker(\partial_1) = \operatorname{rank} \mathrm{CF}_1 = |E|.$$

Since $\mathrm{HF}_1 = \ker(\partial_1)/\mathrm{im}(\partial_2)$,

$$\beta_1 = \operatorname{rank} \ker(\partial_1) - \operatorname{rank} \mathrm{im}(\partial_2) \leq \operatorname{rank} \ker(\partial_1) \leq |E|.$$

The bound $\beta_0 \leq |V|$ follows similarly. □

5.1 Invariance

A key property of Floer homology is independence of auxiliary choices.

Theorem 5.6 (Invariance of HF_*)**.** *The Floer homology* $\mathrm{HF}_*(\mathsf{P})$ *depends only on the isomorphism class of the state-transition graph* $G(\mathsf{P})$*, not on the specific Lagrangian* L *or perturbation data used to construct the chain complex.*

Proof sketch. Two choices L_0, L_1 of Lagrangian satisfying theorem 3.4 are connected by a homotopy L_s, $s \in [0,1]$. The continuation maps induced by this homotopy define chain maps $\Phi : \mathrm{CF}_*^{L_0} \to \mathrm{CF}_*^{L_1}$ and $\Psi : \mathrm{CF}_*^{L_1} \to \mathrm{CF}_*^{L_0}$ whose compositions are chain homotopic to the identity. Hence $\mathrm{HF}_*(L_0) \cong \mathrm{HF}_*(L_1)$. In the discrete setting this reduces to a standard graph homotopy invariance argument. □

6 Main Theorem: Isomorphism with Morse Homology

Theorem 6.1 (Floer-Morse isomorphism)**.** *For any deterministic program* P *with reachable state-transition graph* $G(\mathsf{P})$*, there is a canonical isomorphism of graded* $\mathbb{Z}_2$*-modules:*

$$\mathrm{HF}_*(\mathsf{P}) \;\cong\; \mathrm{HM}_*(G(\mathsf{P})).$$

Proof. The isomorphism is the program-analogue of the *PSS map* (Piunikhin–Salamon–Schwarz) from symplectic Floer theory.

Step 1 (PSS map Φ). Define $\Phi : \mathrm{CF}_k(\mathsf{P}) \to \mathrm{CM}_k(G(\mathsf{P}))$ by the identity on generators: each critical point of $\mathcal{A}$ of Morse index k is identified with the corresponding k-cell of the CW structure on $G(\mathsf{P})$. This is an isomorphism of $\mathbb{Z}_2$-vector spaces.

Step 2 (Boundary compatibility). Under the identification of Step 1, the Floer boundary operator ∂^{Fl} counting gradient-flow trajectories between adjacent critical points coincides with the cellular boundary operator ∂^{cell} of the CW complex. This is because, in the discrete setting, "gradient flow" from an index-k critical point to an index-$(k-1)$ critical point is exactly a transition edge in $G(\mathsf{P})$, and the count (mod 2) of such trajectories is the coefficient of the cellular boundary.

Step 3 (Homology isomorphism). Since Φ is a chain isomorphism (both a vector-space isomorphism and a boundary-compatible map), it induces an isomorphism on homology: $\mathrm{HF}_* \cong \mathrm{HM}_*$. □

Corollary 6.2 (Rank formulas)**.** *For any finite deterministic program* P *with reachable state-transition graph* $G(\mathsf{P}) = (V, E)$*:*

(i) $\operatorname{rank}\mathrm{HF}_0(\mathsf{P})$ *equals the number of connected components of* $\mathrm{Reach}(\mathsf{P})$*.*

(ii) $\operatorname{rank}\mathrm{HF}_1(\mathsf{P}) = |E| - |V| + \operatorname{rank}\mathrm{HF}_0(\mathsf{P})$ *equals the number of independent cycles in* $G(\mathsf{P})$*.*

(iii) The Euler characteristic satisfies $\chi(\mathsf{P}) = |V| - |E| = \beta_0(\mathsf{P}) - \beta_1(\mathsf{P})$*.*

Proof. All three statements follow from theorem 6.1 applied to the standard formulas for graph homology recalled in section 2. □

Corollary 6.3 (Input-independence)**.** *The Betti numbers* $\beta_k(\mathsf{P})$ *are independent of the specific input to the program: they depend only on the topological structure of the reachable state-transition graph. In particular, any two runs that produce isomorphic state-transition graphs have equal Floer homology.*

Example 6.4 (Merge sort)**.** Merge sort on an array of n elements has a balanced recursion tree as its state-transition graph: no cycles, so $\mathrm{HF}_1 = 0$ and $\beta_0 = 1$. *Any* input of the same length produces the same (tree) state-transition graph, confirming input-independence.

7 Application: Deadlock Detection

7.1 Deadlock as a Topological Phenomenon

A program is *deadlocked* at state σ if it reaches σ and then cannot make progress: all enabled transitions lead back into a cycle from which halting states are unreachable. Livelock is the stronger condition that the program cycles forever.

Proposition 7.1 (Deadlock implies cycles)**.** *If a program* P *can deadlock or livelock, then its reachable state-transition graph* $G(\mathsf{P})$ *contains at least one directed cycle.*

Proof. A deadlock or livelock situation requires the program to be in a state σ from which no halting state is reachable. Since the state space is finite, any infinite non-halting execution must revisit some state σ', producing a cycle $\sigma' \to \cdots \to \sigma'$ in $G(\mathsf{P})$. □

Theorem 7.2 (Deadlock detection via Floer homology)**.** *If* $\mathrm{HF}_1(\mathsf{P}) \neq 0$*, then* $G(\mathsf{P})$ *contains a non-trivial directed cycle, and hence* P *is a necessary candidate for deadlock or livelock. Formally:*

$$\mathrm{HF}_1(\mathsf{P}) \neq 0 \implies \exists \text{ a non-contractible cycle in } G(\mathsf{P}).$$

Proof. By theorem 6.1, $\mathrm{HF}_1(\mathsf{P}) \cong \mathrm{HM}_1(G(\mathsf{P}))$. A non-zero element $[c] \in \mathrm{HM}_1(G(\mathsf{P}))$ is represented by a 1-cycle $c \in \mathrm{CM}_1(G(\mathsf{P}))$ with $\partial_1(c) = 0$ that is not the boundary of any 2-chain. A 1-cycle c in the transition graph with $\partial_1(c) = 0$ must visit every vertex an even number of times; in particular, there is at least one vertex visited more than once, giving a directed cycle in $G(\mathsf{P})$. □

Remark 7.3. theorem 7.2 gives only a *necessary* condition for deadlock, not a sufficient one. A program with $\mathrm{HF}_1 \neq 0$ has non-trivial cycles in its state graph, but it may still terminate if every cycle eventually exits to a halting state. The stronger statement—$\mathrm{HF}_1 \neq 0$ *and* no halting state is reachable from the cycle—is a sufficient condition.

7.2 Python Example and JuGeo Integration

The following program implements a mutex acquisition protocol with a potential deadlock when two threads compete for the same resource.

Listing 1: Potential deadlock: two threads competing for a shared mutex.

```
import threading

lock_A = threading.Lock()
lock_B = threading.Lock()

def thread_1():
    with lock_A:
        with lock_B:       # Thread 1 holds A, requests B
            pass

def thread_2():
    with lock_B:
```

```
        with lock_A:       # Thread 2 holds B, requests A
            pass

# State: (owner_A, owner_B) in {None, 1, 2}^2
# Deadlock state: owner_A = 1, owner_B = 2
#   -> Thread 1 waiting for B, Thread 2 waiting for A
# HF_1 detects the cycle:
#   (None,None) -> (1,None) -> (1,2) -> (None,2) -> (None,None)
```

The reachable state-transition graph of this program contains the 4-cycle shown in the comment. By theorem 7.2, $\mathrm{HF}_1 \neq 0$, and JuGeo reports the deadlock condition as a structured obstruction.

Listing 2: JuGeo output for deadlock detection.

```
JuGeo Floer Analysis: thread_1 / thread_2
  Reachable states   : 4  (state = (owner_A, owner_B))
  Transitions        : 4  (mutex acquire/release pairs)
  HF_0 rank          : 1  (one connected component)
  HF_1 rank          : 1  (one independent cycle -> potential deadlock)

  Obstruction [RUNTIME tier]:
    Coordinate    : HF_1 generator
    Violation     : Non-contractible 1-cycle detected in state graph
    Cycle         : (N,N) -[T1 acq A]-> (1,N) -[T2 acq B]-> (1,2)
                    -[T2 rel B]-> (1,N) -[T1 rel A]-> (N,N)
    Repair hint   : Impose consistent lock-acquisition ordering
```

8 Integration with the Judgment Geometry Framework

8.1 Floer Invariants as Trust-Annotated Judgments

The Floer homology groups $\mathrm{HF}_k(\mathsf{P})$ are computed from the reachable state-transition graph, which is itself computed at runtime (or by static analysis). This places Floer computations squarely in the RUNTIME trust tier of the JuGeo framework.

Definition 8.1 (Floer judgment)**.** For a program P and degree $k \in \{0, 1\}$, the *k-th Floer judgment* is the JuGeo 8-tuple:

$$\mathcal{J}_{\mathsf{P}}^{\mathrm{HF}_k} = (\mathsf{HF}_k,\ \beta_k(\mathsf{P}) = r,\ \mathrm{CF}_k(\mathsf{P}),\ [\text{transition graph}],\ \emptyset,\ \mathcal{B}_k,\ \mathsf{RUNTIME},\ G(\mathsf{P})),$$

where $r \in \mathbb{N}$ is the computed Betti number, the evidence bundle contains the transition graph, and the obstruction $\mathcal{B}_k$ is non-empty iff $r > 0$ for $k = 1$ (deadlock candidate).

8.2 The VERIFY Semantic Move

Computing $\mathrm{HF}_k(\mathsf{P})$ is an instance of the Verify semantic move applied to the Floer predicate:

(i) **Extract:** Build $G(\mathsf{P}) = (V, E)$ by executing P on a representative sample of inputs (or by static CFG analysis).

(ii) **Compute:** Evaluate the boundary matrix $M \in \mathbb{Z}_2^{|V|\times|E|}$ and compute $\ker M$ and $\mathrm{im}\, M^T$ via Gaussian elimination over $\mathbb{Z}_2$.

(iii) **Report:** Issue the Floer judgment $\mathcal{J}_{\mathsf{P}}^{\mathrm{HF}_k}$ at trust tier RUNTIME, or escalate to ORACLE if the state space exceeds a configurable bound.

Proposition 8.2 (Complexity of Floer computation)**.** *For a program* P *with* $|V|$ *reachable states and* $|E|$ *transitions, the Floer Betti numbers* β_0, β_1 *can be computed in* $O(|V| \cdot |E|)$ *time via Gaussian elimination over* $\mathbb{Z}_2$*.*

Proof. Gaussian elimination over $\mathbb{Z}_2$ on the $|V| \times |E|$ boundary matrix runs in $O(|V|^2 \cdot |E|)$ in the worst case, but the sparsity of the incidence matrix (each column has exactly 2 non-zero entries) reduces this to $O(|V| \cdot |E|)$ via the standard algorithm for sparse matrices. □

8.3 Connection to Sheaf Cohomology

The sheaf-theoretic approach of earlier papers in the Judgment Geometry series (Papers 3–6) computes Čech cohomology $H^k(\mathcal{S}, \mathcal{F})$ of a sheaf $\mathcal{F}$ over a semantic site $\mathcal{S}$. The Floer homology groups are related by a duality:

Proposition 8.3 (Floer-Sheaf duality)**.** *For a program* P *modelled as a sheaf* $\mathcal{F}_{\mathsf{P}}$ *over the semantic site* $\mathcal{S}_{\mathsf{P}}$*, the Universal Coefficient Theorem over* $\mathbb{Z}_2$ *gives:*

$$\mathrm{HF}_k(\mathsf{P}; \mathbb{Z}_2) \;\cong\; H^k(\mathcal{S}_{\mathsf{P}}, \mathcal{F}_{\mathsf{P}}; \mathbb{Z}_2).$$

This means that obstructions detected by the sheaf-descent machinery (cohomological obstructions to gluing local sections) coincide, over $\mathbb{Z}_2$, with the Floer-homological obstructions to contractibility of state-space cycles. The two frameworks are thus two views of the same underlying geometry.

9 Related Work

Floer theory. Floer homology was introduced in [1] for Hamiltonian diffeomorphisms and extended to Lagrangian intersections, Heegaard splittings, and many other settings; see [2] for a survey. The PSS isomorphism HF ≅ HM is due to Piunikhin–Salamon–Schwarz [3].

Topological program analysis. Algebraic topology has been applied to program analysis via persistent homology [4] (for tracking feature lifetimes in data), and via homotopy-type theory [5] (for reasoning about equality and path spaces in dependent type theory). Our approach differs in applying the full Floer chain complex to the execution trace space, rather than a simpler point-cloud filtration.

Deadlock detection. Classical deadlock detection algorithms include resource allocation graph analysis [6] and Petri net coverability analysis [7]. Our contribution is a topological invariant (HF_1) that detects necessary conditions for deadlock without constructing the full resource allocation graph, and that is composable with the JuGeo trust algebra.

Model checking and cycle detection. LTL/CTL model checking [8] detects liveness violations via cycle detection in the state graph. Our approach is less precise (we detect that *some* cycle exists, not that it violates a specific formula) but more lightweight and compositional, and produces a trust-annotated obstruction rather than a Boolean yes/no answer.

Judgment Geometry series. This paper builds on the earlier papers in the series: the semantic sites of Paper 1 [9], the Čech obstruction machinery of Papers 3–4 [10, 11], and the trust algebra of Paper 5 [12].

10 Conclusion

We have developed Floer homology as a tool for program analysis, identifying execution traces with critical points of an action functional and proving that the resulting Floer chain complex has $\partial^2 = 0$. The main theorem—$\mathrm{HF}_*(\mathsf{P}) \cong \mathrm{HM}_*(G(\mathsf{P}))$—reduces Floer homology to graph homology, giving explicit rank formulas: β_0 counts connected components of reachable state space, and β_1 counts independent cycles.

The principal application is deadlock detection: $\beta_1 > 0$ is a computationally checkable necessary condition for deadlock, which the JuGeo framework surfaces as a RUNTIME-tier structured obstruction. The computation requires only sparse Gaussian elimination over $\mathbb{Z}_2$ and runs in $O(|V| \cdot |E|)$ time.

The Floer and sheaf perspectives are unified by the Universal Coefficient Theorem: cohomological descent obstructions coincide with Floer homological cycle obstructions over $\mathbb{Z}_2$. This suggests a programme of higher Floer invariants—Floer K-theory, Floer cobordism—as sources of progressively finer program invariants, which we leave for future work.

References

[1] A. Floer. Morse theory for Lagrangian intersections. *Journal of Differential Geometry*, 28(3):513–547, 1988.

[2] D. Salamon. Lectures on Floer homology. In *Symplectic Geometry and Topology* (IAS/Park City Mathematics Series), pages 143–229. AMS, 1999.

[3] S. Piunikhin, D. Salamon, and M. Schwarz. Symplectic Floer-Donaldson theory and quantum cohomology. In *Contact and Symplectic Geometry*, pages 171–200. Cambridge University Press, 1996.

[4] H. Edelsbrunner, D. Letscher, and A. Zomorodian. Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4):511–533, 2002.

[5] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013.

[6] E. G. Coffman, M. J. Elphick, and A. Shoshani. System deadlocks. *ACM Computing Surveys*, 3(2):67–78, 1971.

[7] J. Esparza and M. Nielsen. Decidability issues for Petri nets. *Bulletin of the EATCS*, 52:245–262, 1994.

[8] E. M. Clarke and E. A. Emerson. Design and synthesis of synchronization skeletons using branching time temporal logic. In *Workshop on Logic of Programs*, LNCS 131, pages 52–71. Springer, 1981.

[9] H. Young. Semantic sites for program verification. *Judgment Geometry*, Paper 1, 2024.

[10] H. Young. Descent obstructions in program verification. *Judgment Geometry*, Paper 3, 2024.

[11] H. Young. The trust ordered algebra. *Judgment Geometry*, Paper 4, 2024.

[12] H. Young. SMT fragment dispatch. *Judgment Geometry*, Paper 5, 2024.